An FP4 QK^T Attention-Score Accelerator Chiplet on Sky130

Vamsidhar Reddy Eraganeni
ECE 510 – Hardware for AI/ML, Spring 2026
Portland State University
Repository: github.com/vamsireddysup/ECE510-H4AI

***Abstract*—The QK^T score computation in transformer self-attention is a dense matrix multiply whose cost grows quadratically with sequence length. This work presents a co-processor chiplet that performs that multiply using 4-bit floating-point (FP4 E2M1) operands, a 256-entry product lookup table, and FP32 accumulation in an output-stationary systolic array. The synthesized design is a 4×4 array on the SkyWater sky130 high-density process. The RTL-to-GDSII flow completes with zero DRC violations, a clean LVS match, and timing closure at a 15.0 ns clock period, giving a 61.996 MHz suggested operating frequency. At this array size the accelerator is slower than the CPU baseline in wall-clock time but uses about 90 times less energy per operation. A quantization study shows that the FP4 multiply is exact through a full product table, that per-element score error is large, and that the resulting attention distribution is far more robust than the raw scores because softmax depends on score ranking. A 16×16 array is the scaling target; all 16×16 figures are projections, not measurements.**

***Index Terms*—systolic array, attention, FP4, microscaling, sky130, OpenLane, roofline**

I. Problem and Motivation

Transformer self-attention [1] multiplies a query matrix Q by a transposed key matrix K^T for every layer and head. The product is a dense matrix multiply, and its cost grows with the square of the sequence length, so it takes a larger share of inference time as context windows get longer. I set out to build a chiplet that does this one operation faster than a general-purpose CPU.

The M1 profiling fixed the baseline I am measuring against. On an Intel Core i5-1145G7 (Tiger Lake, 4 cores) running PyTorch 2.11 on the CPU in float32 at batch size one, QK extsuperscriptT at sequence length $T = 512$ reaches 11.36 GFLOP/s, with a kernel latency of 2.954 ms for the $N = 512$, $d_{head} = 64$ case and a memory bandwidth of 51.2 GB/s. The measurement used the default PyTorch CPU backend with three warmup runs and the median of ten timed runs; thread count was not pinned, so this is PyTorch's default CPU execution rather than a thread-restricted single-core run. The workload is the extttMultiHeadAttention forward pass from a public reference transformer implementation, with the scaled-dot-product step isolated as the kernel of interest. Profiling put roughly half of attention time in the scaled dot-product step. The CPU hits only a few percent of its floating-point peak here because it does not have enough multiply units to keep its datapath busy, and because fetch, decode, and cache traffic burn cycles that a fixed-function datapath would not. Latency also grew faster than linearly with sequence length, so the quadratic term is already showing at $T = 512$.

Specialized matrix engines answer this by spending area on multiply-accumulate hardware and reusing operands on chip. Google's TPU [2] is the well-known example: a large systolic array of multiply-accumulate cells that keeps data moving between neighbors so each operand feeds many operations before it leaves the array. Low-precision number formats push in the same direction by making each multiply cheaper, and recent work on microscaling formats [5] pairs a narrow element format with a shared scale factor per block to recover some of the range that a 4-bit mantissa gives up. The design here borrows both ideas at a scale that fits a one-term course project: a small systolic array of FP4 multiply-accumulate elements with a per-row and per-column scale, built and pushed through an open RTL-to-GDSII flow on the sky130 process. Whether that pays off at the array size I can actually synthesize is the question the rest of this report works through.

II. Roofline Analysis

The roofline model [3] tells me whether the kernel is limited by compute or by data movement. Arithmetic intensity (AI) is the ratio of floating-point operations to bytes moved. For QK^T with $N = 512$ and $d_{head} = 64$, the operation count is

$$\text{FLOPs} = 2 \cdot N^2 \cdot d_{head} = 2 \cdot 512^2 \cdot 64 = 33.55 \text{ M}. \quad (1)$$

Byte traffic depends on how much data the hardware reuses, which gives two bounds. With no reuse, every output element reloads its operand rows, which moves about 17 MB and gives a lower bound of

$$\text{AI}_{\text{low}} = \frac{33.55 \times 10^6}{17.83 \times 10^6} = 1.88 \text{ FLOP/byte}. \quad (2)$$

With full on-chip reuse, each row of Q and K loads once and only the output is written, which moves about 1.03 MB and gives an upper bound of

$$\text{AI}_{\text{high}} = \frac{33.55 \times 10^6}{1.08 \times 10^6} = 31.03 \text{ FLOP/byte}. \quad (3)$$

Because FP4 operands are so small, the FP32 output write ends up dominating the byte count at the upper bound. This is worth noting on its own: shrinking the inputs to 4 bits moves the kernel toward being output-write limited, not input-read limited, which is the opposite of the usual full-precision case.

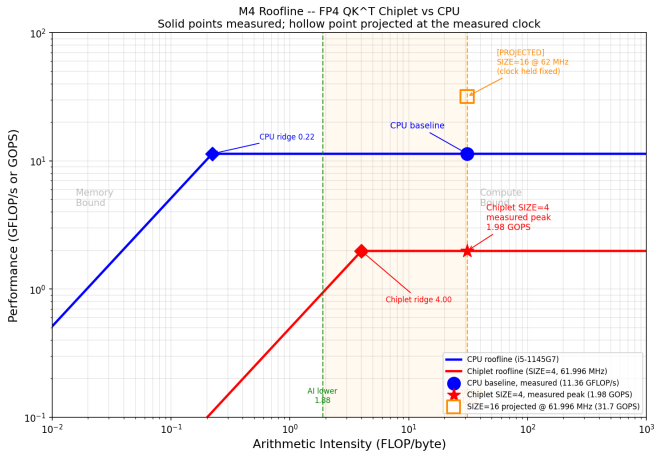

Fig. 1. Roofline for the CPU baseline and the FP4 chiplet. The shaded band is the kernel arithmetic-intensity range from no-reuse (1.88) to full-reuse (31.03). The measured `SIZE=4` point sits below the CPU; the `SIZE=16` points are projected.

The CPU ridge point is its peak over its bandwidth, $11.36/51.2 = 0.22$ FLOP/byte. The chiplet ridge point at the synthesized operating point is its 1.984 GOPS peak over its 0.496 GB/s stream bandwidth, 4.02 FLOP/byte. At the upper-bound intensity of 31.03, the kernel sits well to the right of both ridge points, so it is compute-bound on both platforms. That is the result that drives the architecture: what limits performance is the number and speed of the multiply-accumulate units, not memory bandwidth, and that is the case a systolic array is meant to handle. Fig. 1 plots both rooflines, the intensity band, and the operating points.

Two design choices follow from this. Because the kernel is compute-bound, the area goes into processing elements rather than large caches or wide memory ports. And because the upper bound depends on reuse, the dataflow keeps operands moving through the array so they are read once and used many times.

III. Precision and Data Format

The operands use FP4 in the E2M1 encoding: one sign bit, two exponent bits, one mantissa bit. That gives sixteen codes for fifteen distinct values, the magnitudes $\{0, 0.5, 1.0, 1.5, 2.0, 3.0, 4.0, 6.0\}$ with both signs. The spacing is not uniform. From 0 to 2.0 the step is 0.5, from 2.0 to 4.0 it is 1.0, and from 4.0 to 6.0 it is 2.0. The format has fine resolution near zero and coarse resolution near its top, which matters for the error analysis below.

The multiplier is a 256-entry lookup table indexed by the two 4-bit operands concatenated together. Each entry holds the exact IEEE-754 FP32 encoding of the product. With only $16 \times 16 = 256$ possible operand pairs, the table stores every product exactly, so the multiply introduces no rounding error. The only place precision is lost is when the original Q and K values get encoded into FP4 before they reach the chip. Once inside, the product and the FP32 accumulation are exact with respect to those FP4 inputs. Accumulation runs in IEEE-754 FP32 through a three-stage pipelined adder, which avoids the error that would build up if partial sums were cut back to a narrow format between steps.

The interesting question is what the FP4 input encoding costs. I measured this with a standalone study: draw Q and K as Gaussian matrices, clip them into the FP4 range, round each element to the nearest representable FP4 value, and compute QK^{T} with the exact multiply-accumulate the hardware uses. Comparing against the full-precision product on the same clipped inputs isolates the input-quantization error. Table I shows the result for several input scales. The per-element error is large: the median relative error on the scores stays in the 15 to 26 percent range, because fifteen values cannot represent a continuous input finely, and almost no input lands exactly on a grid point. Rounding a value near the midpoint of the largest gap, around 5.0, costs a full 20 percent on that element alone.

TABLE I
FP4 input-quantization error on QK^{T} (N=512, d=64), inputs $\sim \mathcal{N}(0, \sigma)$ clipped to the FP4 range, exact FP32 MAC.

σ	mean $\lvert e \rvert$	RMS $\lvert e \rvert$	median rel. err.
0.5	0.66	0.83	26.1%
1.0	1.38	1.73	18.7%
1.5	2.43	3.06	15.9%
2.0	3.95	4.99	15.0%

On its own that error looks disqualifying. What rescues it is that attention does not use the raw scores; it uses the softmax over them, which depends on the ranking and the gaps between the largest scores rather than their exact values. On the same σ=1.0 data, the attention distribution computed from the FP4 scores is close to the full-precision one: the mean Kullback-Leibler divergence between the two softmax distributions is 0.023 nats and the mean difference in attention weight per entry is 0.0003. The weak spot is the top-1 position, where the FP4 scores agree with the reference argmax only 62 percent of the time, and the top-5 overlap is 73 percent. This matters more in some uses than others. Where attention feeds a soft weighted sum over values, a swapped top-1 between two nearly tied scores costs little, because the two were close to begin with and the softmax weights them similarly, which is what the low KL divergence reflects. But in settings that take a hard argmax over attention, such as some retrieval or greedy-selection schemes, a 62 percent top-1 match is a real degradation, not a rounding detail. The format would need either a finer encoding or tighter input scaling for those uses. The Gaussian stress test here is a worst case in that respect, since it spreads inputs across the full FP4 range where the coarse high-magnitude steps do the most damage.

This is why the design carries a per-row and per-column scale factor in `scale_sram.sv`. The Gaussian test above is a deliberate stressor; real attention inputs are layer-normalized and better behaved, and a scale factor lets each row and column be mapped into the part of the FP4 range where the 0.5 steps give the finest resolution. The honest summary is that FP4 is a coarse format whose per-element error is high, that

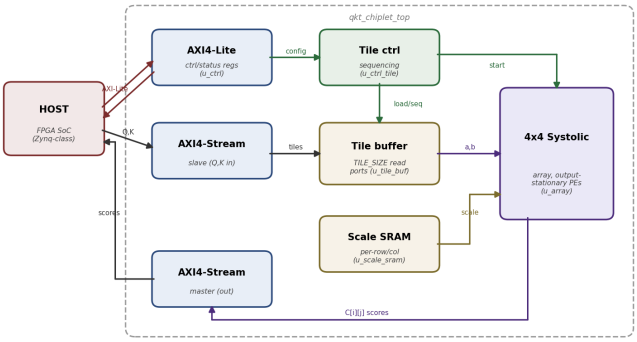

Fig. 2. Top-level block diagram of `qkt_chiplet_top`. AXI4-Lite carries control and status; AXI4-Stream carries Q and K in and the scores out. The tile controller sequences the tile buffer and the array, the scale SRAM holds the per-row and per-column factors, and the 4×4 systolic array does the multiply-accumulate.

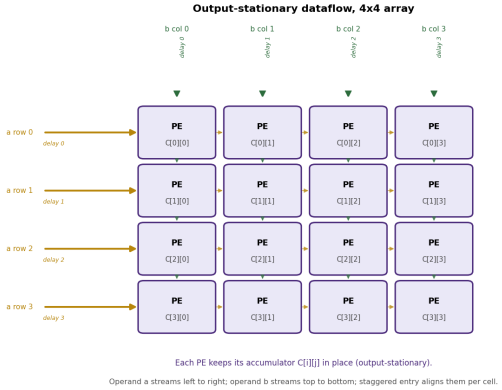

Fig. 3. Output-stationary dataflow. Operand a streams left to right and operand b top to bottom; each row and column is delayed by its index so the operands that meet at PE (i, j) are the ones for output $C[i][j]$, which stays resident in that PE.

the multiply and accumulation add nothing to that error, and that the softmax that consumes the scores is tolerant enough for the format to be usable when the inputs are scaled sensibly. The end-to-end verification in Section VI uses small integer-valued test inputs that land exactly on FP4 grid points, so the hardware reproduces the reference result with no error and isolates functional correctness from the quantization question studied here.

IV. Dataflow and Architecture

The array follows the classic systolic-array idea [4]: a regular grid of simple cells, each talking only to its neighbors. The array is output-stationary. Each processing element (PE) owns one accumulator that holds one output element $C[i][j]$ and keeps it in place for the whole computation, while operands flow through the array between neighbors. The RTL does exactly this: each PE registers and forwards its operands (`a_out <= a_in`, `b_out <= b_in`) through stagger registers and accumulates its own result locally. Output-stationary fits QK^{T} well because each score is an independent dot product over the head dimension, so holding the output fixed and streaming the contracted dimension through it is the natural mapping, and it gives the reuse the roofline upper bound assumes.

Operands enter the array on staggered diagonals. Row i of the a operands is delayed by i cycles through a chain of stagger registers, and the same is done column-wise for the b operands, so that the operands meeting at PE (i, j) are the ones that belong to output $C[i][j]$. Figure 3 shows this. The staggering is what lets a single broadside of inputs feed the whole array without a crossbar, and it is the reason the first valid output appears several cycles after the last input is presented rather than immediately.

Inside each PE the work is collect-then-reduce, not a fused multiply-accumulate every cycle. The PE first computes all d_{head} products through the table and writes them into a local buffer, then runs a sequential loop that feeds the FP32 adder one product at a time. Because the adder has a three-cycle latency, the PE waits three cycles per accumulation step before it takes each partial sum, so a single dot product over d_{head} elements takes roughly $3\,d_{head}$ cycles in the accumulation phase. This is a deliberate simplicity-for-speed trade: a single shared pipelined adder per PE is small and easy to verify, but it serializes the reduction and is the main reason throughput is low. A fused multiply-add tree would cut the reduction to a logarithmic depth at a large area cost; for a course project the serial reduction was the safer choice.

The cost shows up directly in the cycle counts from the end-to-end run on a single 4×4 tile with $d_{head} = 4$: the first output becomes valid 140 cycles into the run, and the whole tile completes in 498 cycles. Most of those cycles are accumulation and drain, not input streaming, which is consistent with the serial reduction inside each PE being the bottleneck rather than the array fill.

The compute core is parameterized by `SIZE`. The synthesized and benchmarked design uses `SIZE=4`, sixteen PEs in a 4×4 grid. The `SIZE=16` configuration is the target and is treated as a projection everywhere it appears. The top level wraps the array with an AXI4-Lite control interface, an AXI4-Stream data interface, and a tiling controller that sequences operand loading, computation, and result readback. For synthesis the array goes through a flat-port wrapper, `systolic_array_flat`, because the synthesis front-end could not parse the unpacked-array ports of the original module. I explain that in Section IX.

V. Hardware Interface

Control uses AXI4-Lite, a light register interface for writing configuration such as the matrix size and for polling a status register. Bulk data uses AXI4-Stream, a valid/ready handshaked interface that carries the Q and K operands in and the FP32 scores out. The stream path is 64 bits wide and carries packed operands per beat. The handshake means the chiplet stalls cleanly when the host is not ready to send or

receive, so the interface does not need to know the internal timing of the array.

At 61.996 MHz a 64-bit stream moves $8 \times 61.996 \times 10^6 = 0.496$ GB/s. The roofline puts the kernel in the compute-bound region at the reuse the array gets, so this is enough bandwidth and the interface is not the bottleneck at `SIZE=4`. The limit at this size is how fast the small array can retire accumulations; operands arrive faster than the array can consume them. If the array were scaled up and the accumulation pipelined harder, the balance would shift and interface bandwidth would start to matter, but at this size there is margin to spare. I split control and data onto two interfaces precisely so that the low-rate configuration traffic and the high-rate operand stream do not contend, which keeps the stream path simple and full-throughput.

VI. Verification

I checked correctness at two levels. At the unit level, the M2 testbenches exercise the blocks one at a time. The FP4 multiplier test sweeps operand pairs and checks each product against the expected FP32 value, which validates the table contents. The PE tests include a small hand-checked case, a $d_{head} = 64$ configuration that stresses the full product buffer and the longer accumulation loop, and a randomized-operand test that compares the PE output against a software dot product. The systolic-array test compares the whole array against a golden reference matrix multiply. The AXI4-Lite test writes and reads back control registers and checks the handshake, the AXI4-Stream test pushes operands through the valid/ready interface, and the tiling-controller test checks the sequencing of load, compute, and readback. These are C++ Verilator testbenches under `project/m2/tb/`.

At the system level, an end-to-end co-simulation drives the full chiplet through a complete transaction: configuration writes over AXI4-Lite, streaming of the scale factors and the Q and K tiles over AXI4-Stream, the systolic computation, and readback of the scores. The test uses small integer-valued inputs chosen so that every operand lands exactly on an FP4 grid point, which removes quantization from the comparison and tests only that the datapath and control are functionally correct. The expected result for the 4×4 case is a symmetric matrix with 2.0 on the diagonal and a mix of 0.0 and 1.0 off it. The log in `project/m4/sim/final_run.log` shows the hardware reproducing this matrix on all sixteen elements, with the output going valid 140 cycles in and the tile finishing in 498 cycles. Fig. 4 is the annotated waveform of that run, with the host-write, compute, and host-read phases marked and the handshake signals labeled. The exact match on all sixteen elements confirms the table contents, the staggered operand routing, and the accumulation order together.

VII. Synthesis Results

Synthesis and the physical flow ran in OpenLane on the sky130 high-density library. The flow finished with status success: zero Magic DRC violations, zero KLayout DRC violations, a clean LVS match across 43,158 nets, and no XOR differences between the Magic and KLayout layouts.

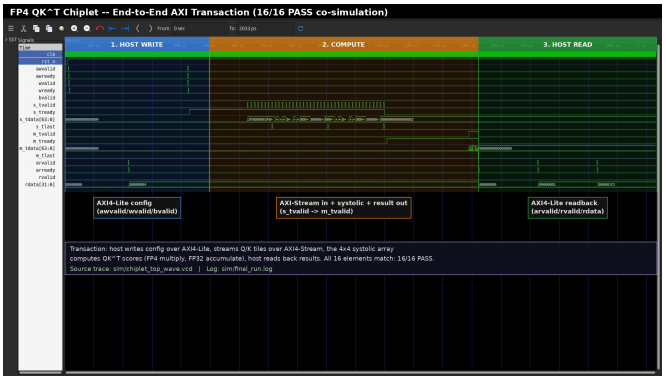

Fig. 4. Annotated end-to-end co-simulation waveform showing the host-write, compute, and host-read phases of one QK^T transaction. All sixteen output elements match the reference (16/16 PASS).

Timing closed at a 15.0 ns clock period. The critical path runs through the FP32 accumulate logic and measures 6.19 ns of logic delay. That path goes from an accumulator flip-flop, through the operand select into the FP32 adder, through the adder's exponent-compare, mantissa-align, and add stages, and back to the accumulator register. The remaining gap between 6.19 ns and the 15 ns period is clock-tree skew and uncertainty on the single-clock flat netlist rather than logic delay, which is why relaxing the period closed the design even though the logic itself is well under 7 ns. The flow's setup and hold gates pass, and the suggested operating frequency is 61.996 MHz. The nominal-corner SPEF report still shows a small negative setup slack of about -1.13 ns, which is the skew margin I discuss in Section IX, so I report 62 MHz as the frequency the design is safe at rather than the 66.7 MHz the period nominally implies.

Synthesis produced 30,689 standard cells over 324,753 μm^2; after place and route the cell count is 81,750 over a core area of 589,240 μm^2. The largest area contributor is sequential logic, with 5,702 D flip-flops that hold the per-PE product buffers and 32-bit accumulators, plus the FP32 datapath and about 1,600 multiplexers in the accumulation addressing. The flip-flop count is a direct consequence of the collect-then-reduce PE: buffering all d_{head} products before reducing them costs registers, and that choice shows up here as the dominant area term. Typical-corner power is 28.1 mW, split 54.2% sequential, 40.5% clock, and 5.3% combinational. The large clock share matches a flip-flop-heavy design and shows that the registers and the clock tree that drives them, not the combinational arithmetic, set both area and power.

VIII. Benchmark Results

Throughput comes from the synthesized frequency and the cycle behavior from simulation. At `SIZE=4` and 62 MHz the array peaks at $62 \times 10^6 \times 4 \times 4 \times 2 = 1.984$ GOPS. Against the CPU baseline of 11.36 GFLOP/s, the accelerator at this size

is slower in raw throughput. The 4×4 array does not beat the CPU on wall-clock time: sixteen PEs at 62 MHz cannot match a 2.6 GHz superscalar core with SIMD, and the three-cycle per-step accumulation in each PE widens the gap.

Two things cause this. Peak throughput scales with the square of the array dimension, so `SIZE=4` uses only a small fraction of the parallelism the design is meant to have. And the collect-then-reduce PE with a three-cycle adder spends many cycles per output, so utilization stays low. Figure 1 shows the measured `SIZE=4` point sitting below the CPU.

Energy is where the design comes out ahead. At 28.1 mW the chiplet draws far less power than the CPU, so even with worse latency the energy per QK^T is lower, around 90× lower at the ideal-latency estimate. Peak power efficiency is about 70.6 GOPS/W. At this scale that efficiency, rather than wall-clock speed, is what the design has to offer, and it comes directly from the FP4 multiply being a small table lookup instead of a full floating-point multiplier.

Getting competitive latency needs a larger array. Scaling the measured peak with PE count, a 16×16 array at the same 62 MHz clock the SIZE=4 design actually closed would reach about 31.7 GOPS, a projected 2.79× speedup over the CPU. This projection holds the clock fixed at the frequency I measured and only scales the parallelism, so it does not assume any timing improvement. A higher clock such as the original 250 MHz target is not projected here: the SIZE=4 design failed to close at 250 MHz (Section IX), so quoting a 250 MHz number for a larger array would not be supported by anything I measured. Reaching a higher clock depends on the FP32-adder pipelining listed as future work in Section IX. The 62 MHz projection assumes throughput scales with PE count and that the per-PE reduction is pipelined enough not to stall a larger array, which is itself an open item. The numbers behind every value here are in `project/m4/bench/benchmark_data.csv`.

IX. What Did Not Work

A few things went wrong and shaped the result.

The original systolic array used unpacked-array ports, which the synthesis front-end would not parse. The fix was the flat-port wrapper, `systolic_array_flat`, that packs and unpacks the bit vectors at the boundary. That is why the synthesized module differs from the original behavioral one, and why the AXI and tiling logic are checked in co-simulation rather than carried through this synthesis run. The consequence is that the synthesis numbers cover the compute core, and the interface logic is verified functionally rather than placed and routed.

The newer OpenLane 2 flow broke on a Python and yosys version mismatch, so I fell back to OpenLane 1.1.1 in a container. That cost time but gave a clean signoff.

Timing closure at the original 250 MHz target failed. At a 4.0 ns clock the post-route setup slack was badly negative, around -8.49 ns, because the FP32 accumulate path plus clock-tree skew is far longer than 4 ns. I relaxed the period step by step, watching the post-route slack each time, until the flow closed at 15.0 ns. A small nominal-corner negative slack of about -1.13 ns still remains there, which is why I report 61.996 MHz rather than the 66.7 MHz the 15 ns period nominally implies. Hold violations showed up during this process and were fixed by raising the hold-fix resizer margins, which insert buffers on the fast paths.

One correction belongs here too. The M1 Heilmeier write-up gave a CPU ridge point of 8.28 FLOP/byte and a kernel intensity of 25.60 FLOP/byte. Re-deriving the ridge from the measured numbers, $11.36/51.2$, gives 0.22 FLOP/byte, so the M1 value was wrong. The roofline used here and in the CF09 analysis uses 0.22 for the CPU ridge and the 1.88 to 31.03 reuse range for the kernel. The conclusion that the kernel is compute-bound does not change, but the numbers are now right, and finding the discrepancy was a useful check on the earlier work.

The quantization study in Section III also did not go the way I first expected. I had assumed FP4 would be close to lossless on the scores; it is not, with per-element errors well into the double digits in percent. The result that made the format defensible was the softmax robustness, which I only checked after the raw-score numbers came out worse than expected. That changed how I describe the format: not as near-lossless, but as coarse and softmax-tolerant when scaled.

With more time, the changes worth making are to pipeline the FP32 adder so the critical path drops below 4 ns and a shorter clock period closes, to add a reset buffer tree for the high-fanout reset that caused the fanout warnings, and to build the `SIZE=16` array so the projected throughput becomes measured. These are listed in `project/remaining_tasks.md`.

Acknowledgment

Developed for ECE 510, Hardware for AI/ML, Portland State University, Spring 2026. Tools: Verilator, OpenLane 1.1.1, the SkyWater sky130 PDK, and yosys.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[2] N. P. Jouppi *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annu. Int. Symp. Computer Architecture (ISCA)*, Toronto, ON, Canada, 2017, pp. 1–12.

[3] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.

[4] H. T. Kung, "Why systolic architectures?" *IEEE Computer*, vol. 15, no. 1, pp. 37–46, Jan. 1982.

[5] B. D. Rouhani *et al.*, "Microscaling data formats for deep learning," 2023, arXiv:2310.10537.